

Лабораторная работа № 3 по курсу дискретного анализа: исследование качества программ

Выполнил студент группы М8О-208Б-22 МАИ *Немкова Анастасия*.

Условие

Для реализации словаря из предыдущей лабораторной работы, необходимо провести исследование скорости выполнения и потребления оперативной памяти. В случае выявления ошибок или явных недочётов, требуется их исправить.

Результатом лабораторной работы является отчёт, состоящий из:

- Дневника выполнения работы, в котором отражено что и когда делалось, какие средства использовались и какие результаты были достигнуты на каждом шаге выполнения лабораторной работы.
- Выводов о найденных недочётах.
- Сравнение работы исправленной программы с предыдущей версией.
- Общих выводов о выполнении лабораторной работы, полученном опыте.

Минимальный набор используемых средств должен содержать утилиту `gprof` и библиотеку `dmalloc`, однако их можно заменять на любые другие аналогичные или более развитые утилиты (например, `Valgrind` или `Shark`) или добавлять к ним новые (например, `gcov`).

Метод решения

Для анализа качества программ я использовала две утилиты: `valgrind` и `gprof`.

Valgrind - это инструментальное программное обеспечение, предназначенное для отладки использования памяти, обнаружения утечек памяти и профилирования.

При запуске `Valgrind` используются различные флаги, такие как `-leak-check=full`, который проверяет утечки памяти, `-track-origins`, который позволяет отслеживать происхождение ошибок, и `-tool=memcheck`, который запускает основной инструмент `Memcheck` для обнаружения проблем памяти.

Gprof - это утилита профилирования для анализа производительности программ в UNIX-подобных операционных системах. Она позволяет определить, сколько времени программа проводит в каждой функции и как часто эти функции вызываются.

Описание программы

Для запуска утилиты `Valgrind` скомпилируем программу и пропишем в консоли `valgrind -tool=memcheck -leak-check=full ./a.out`

```

arnemkova@LAPTOP-TA2RV74U:~/DA_labs$ valgrind --tool=memcheck --leak-check=full ./a.out
==100== Memcheck, a memory error detector
==100== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==100== Using Valgrind-3.15.0 and LibVEX; rerun with -h for copyright info
==100== Command: ./a.out
==100==
==100==
==100== HEAP SUMMARY:
==100==     in use at exit: 122,880 bytes in 6 blocks
==100==   total heap usage: 1,609,393 allocs, 1,609,387 frees, 411,797,786 bytes allocated
==100==
==100== LEAK SUMMARY:
==100==    definitely lost: 0 bytes in 0 blocks
==100==    indirectly lost: 0 bytes in 0 blocks
==100==    possibly lost: 0 bytes in 0 blocks
==100==    still reachable: 122,880 bytes in 6 blocks
==100==         suppressed: 0 bytes in 0 blocks
==100== Reachable blocks (those to which a pointer was found) are not shown.
==100== To see them, rerun with: --leak-check=full --show-leak-kinds=all
==100==
==100== For lists of detected and suppressed errors, rerun with: -s
==100== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

В данном выводе HEAP SUMMARY показывает, что всего 122,880 байт в 6 блоках были использованы программой при её завершении. Общий объем памяти, выделенной и освобожденной программой, составляет 411,797,786 байт.

LEAK SUMMARY показывает, что не было обнаружено утечек памяти в трех категориях: "definitely lost" (определенно потерянная), "indirectly lost" (косвенно потерянная) и "possibly lost" (возможно потерянная). Однако было обнаружено, что 122,880 байт в 6 блоках все еще доступны (still reachable). Использование флага `--show-leak-kinds=all` показало, что это происходит из-за функции `std::ios_base::sync_with_stdio(bool)`, которая отключает синхронизацию `iostream` с `stdio`.

ERROR SUMMARY показывает, что не было обнаружено ошибок памяти в программе.

Для запуска утилиты Gprof:

- Компилируем программу с флагом профилирования `-pg`
- Исполняем программу для создания файла с профилировочной информацией `gmon.out`
- Запускаем `gprof` для анализа созданного файла

Диагностика проводилась на тесте из 100- строк с операциями вставки, удаления и поиска

```

arnemkova@LAPTOP-TA2RV74U:~/DA_labs$ g++ -pg lab2.cpp
arnemkova@LAPTOP-TA2RV74U:~/DA_labs$ ./a.out

```

arnemkova@LAPTOP-TA2RV74U:~/DA_labs\$ gprof a.out -p gmon.out

Flat profile:

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self us/call	total us/call	name
95.51	0.82	0.82	1529384	0.54	0.54	TString::TString(TString const&)
2.33	0.84	0.02	40001	0.50	0.50	TString::operator=(TString const&)
1.16	0.85	0.01	30001	0.33	0.33	TString::TString(char const*)
1.16	0.86	0.01	30000	0.33	0.33	RandString(char*)
0.00	0.86	0.00	1569387	0.00	0.00	TString::~~TString()
0.00	0.86	0.00	520108	0.00	0.00	TAVL<TString, unsigned long ↳ long>::GetHeight(TAVL<TString, unsigned long long>::TNode*)
0.00	0.86	0.00	513315	0.00	1.07	operator>(TString const&, TString ↳ const&)
0.00	0.86	0.00	251377	0.00	1.07	operator<(TString const&, TString ↳ const&)
0.00	0.86	0.00	134589	0.00	0.00	TAVL<TString, unsigned long ↳ long>::FixHeight(TAVL<TString, unsigned long long>::TNode*)
0.00	0.86	0.00	125465	0.00	0.00	TAVL<TString, unsigned long ↳ long>::BFactor(TAVL<TString, unsigned long long>::TNode*)
0.00	0.86	0.00	120873	0.00	0.00	TAVL<TString, unsigned long ↳ long>::Balance(TAVL<TString, unsigned long long>::TNode*)
0.00	0.86	0.00	30000	0.00	20.93	TAVL<TString, unsigned long ↳ long>::FindNode(TAVL<TString, unsigned long long>::TNode*, TString const&)
0.00	0.86	0.00	10002	0.00	0.00	TString::TString()
0.00	0.86	0.00	10000	0.00	19.35	TAVL<TString, unsigned long ↳ long>::InsertNode(TAVL<TString, unsigned long long>::TNode*, TAVL<TString, unsigned ↳ long long>::TNode*)
0.00	0.86	0.00	10000	0.00	20.93	TAVL<TString, unsigned long ↳ long>::Find(TString const&)
0.00	0.86	0.00	10000	0.00	0.50	TAVL<TString, unsigned long ↳ long>::TNode::TNode(TString const&, unsigned long long const&)
0.00	0.86	0.00	10000	0.00	0.00	TAVL<TString, unsigned long ↳ long>::TNode::~~TNode()
0.00	0.86	0.00	10000	0.00	40.78	TAVL<TString, unsigned long ↳ long>::Insert(TString const&, unsigned long long const&)
0.00	0.86	0.00	10000	0.00	20.93	TAVL<TString, unsigned long ↳ long>::Remove(TString const&)
0.00	0.86	0.00	3448	0.00	0.00	TAVL<TString, unsigned long ↳ long>::RotateRight(TAVL<TString, unsigned long long>::TNode*)
0.00	0.86	0.00	3410	0.00	0.00	TAVL<TString, unsigned long ↳ long>::RotateLeft(TAVL<TString, unsigned long long>::TNode*)
0.00	0.86	0.00	1	0.00	0.00	_GLOBAL__sub_I_ZN7TStringC2Ev
0.00	0.86	0.00	1	0.00	0.00	↳ __static_initialization_and_destruction_0(int, int)
0.00	0.86	0.00	1	0.00	0.00	TAVL<TString, unsigned long ↳ long>::Delete(TAVL<TString, unsigned long long>::TNode*)
0.00	0.86	0.00	1	0.00	0.00	TAVL<TString, unsigned long ↳ long>::TAVL()

```

0.00      0.86      0.00      1      0.00      0.00  TAVL<TString, unsigned long
↪ long>::~TAVL()

```

Данный вывод показывает, что программа большую часть времени тратит на обработку ключей, а не на операции с деревом. 95.51% времени работы программы затрачивается на конструирование объектов типа TString. Далее, 2.33% времени занимает оператор присваивания TString::operator=(TString const).

В целом, чтобы улучшить производительность, следует оптимизировать работу с ключами и операции копирования и конструирования строк типа TString.

Выводы

В ходе данной лабораторной работы я изучила средства диагностики и профилирования программ. В частности, я познакомилась с утилитами valgrind и gprof, которые позволяют выявлять утечки памяти, анализировать использование ресурсов и оптимизировать производительность программы.

Эти утилиты играют важную роль в разработке программ, помогая оптимизировать код с целью уменьшения потребляемой памяти и времени работы программы.